

Learning Objectives:

By the end of this module, you should be able to:

- Get an overview of AWK
- Performs pattern matching within files using AWK
- Create AWK scripts to generate reports from text files

Overview of AWK

AWK is a scripting language supported by most of the Linux distributions. AWK supports text processing and is useful in report generation. It is generally used for data extraction from flat files. AWK usually works on streams of textual data to perform set of actions for data extraction. AWK is widely used to generate reports from flat files or from log files. Let us understand the applicability of AWK in networking domain. Suppose, we are using NS2 simulator that simulates large network. This simulator will generate trace files which contain the records of each packet transfer happening in the network. The amount of records generated by NS2 can be in thousands, when we want to generate a summary of the packet transfers, AWK can be used to extract certain records according to user's criteria. AWK is a widely used tool for reporting purpose.

AWK was created in 1977 at Bell Labs. The name, AWK is derived from the beginning letters of the surnames of its developers/authors who were Alfred Aho, Peter Weinberger and Brian Kernighan. Later, during 1985-88, AWK was revised by Paul Rubin, Jay Fenalson and Richard Stallman. The revised AWK version was released in 1988 as GNU AWK. As of now, AWK is available in various Linux distributions, AWK has many variants like gawk, mawk, nawk etc.

File Structure and AWK

AWK reads from a file or from the standard input. The output of AWK is to its standard output. AWK adheres to the concepts of "file", "record" and "field" as used in flat file model. A structured text file consists of records, usually, each line of a file represent a single record. So, each line is a record for AWK. AWK operates on one record at a time. A record consists of field(s) which are structured as columns, these fields are separated by space(s) or tab(s) by default. Each field can be accessed by prefixing „\$“ sign, so the first field can be accessed as \$1, field 2 with \$2, and so on, however, the field \$0 refers to the whole record. Hence a structure file containing rows are identified as records and

columns are identified as fields. The following figure is an example of flat file structure where rows are identified as different records and columns are identified as fields:

Flat File Model

	Route No.	Miles	Activity
Record 1	I-95	12	Overlay
Record 2	I-495	05	Patching
Record 3	SR-301	33	Crack seal

Figure 1: Flat File Model
(Source: https://commons.wikimedia.org/wiki/File:Flat_File_Model.svg)

Let us take an example to understand the structure of file. Suppose a system with /etc/passwd file contains the following entries in its tail section:

```
shahrukh:x:504:504::/home/shahrukh:/bin/bash
akshay:x:505:505::/home/akshay:/bin/bash
salman:x:506:506::/home/salman:/bin/bash
```

AWK will see this file as follows:

1 line indicates 1 record, so in total there are 3 records in the above mentioned file. 1 record indicates 7 fields in the above case, the records are separated by ":". It must be noted that by default, the field separator is space.

AWK is a programming language. AWK uses specific structure of program to perform pattern matching. An awk program is a sequence of statements that take the following form:

```
pattern { action }
pattern { action }
pattern { action }
.....
```

In general, the format is as follows:

```
awk options "selection criteria {actions}" file(s)
```

Where, pattern acts as a selector that determines whether the action is to be executed. Patterns can be: arithmetic expressions, relational expressions, regular expressions string-valued or boolean expressions. These actions can be a

sequence of statements terminated by newlines or semicolons. AWK programs can either be executed on command prompt or written in a file. The following figure illustrates use of pattern matching in AWK:

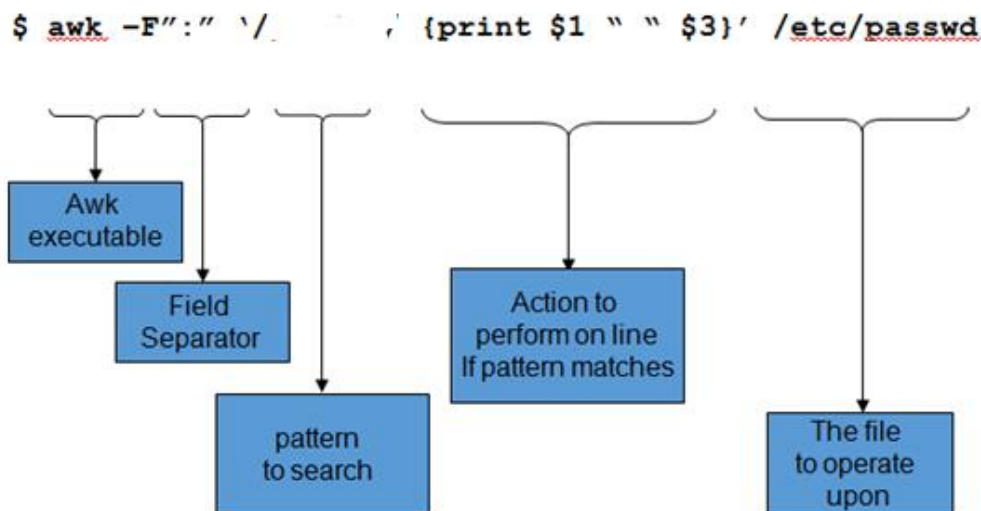


Figure 2: AWK patterns and field

Now, let us see how AWK can be used to fetch various patterns from the file.

AWK Command to fetch the userid of user "akshay":

```
$awk -F":" "/akshay/ {print $1 " " $3}" /etc/passwd
```

Output:

akshay 504

Another way to write the command is:

```
$awk "BEGIN { FS=\":\" } / akshay / {print $1 " " $3}" /etc/passwd
```

Output:

akshay 504

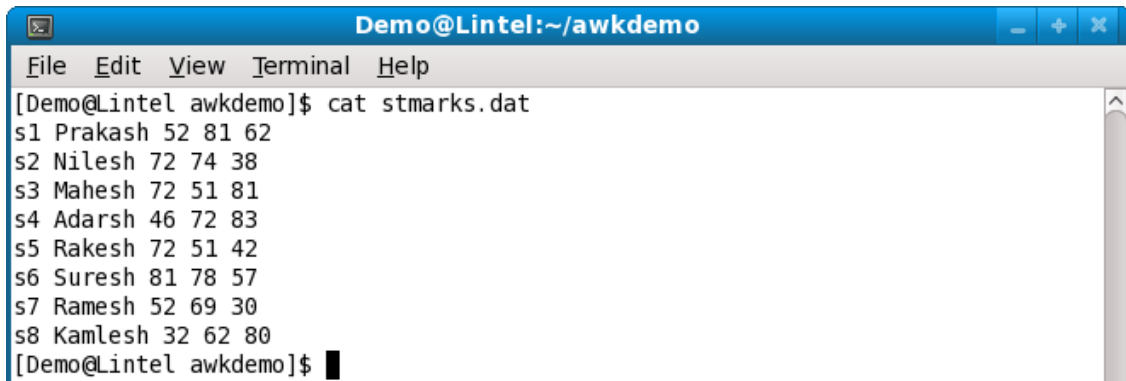
In the above example, print command is used to output text on standard output. The output text is always terminated with a newline. Here, the „F” option is used to specify field separator which is ":". The print command can be used as follows to display the first field:

```
print $1
```

The following command displays the first field and third field of the current record:

```
print $1, $3
```

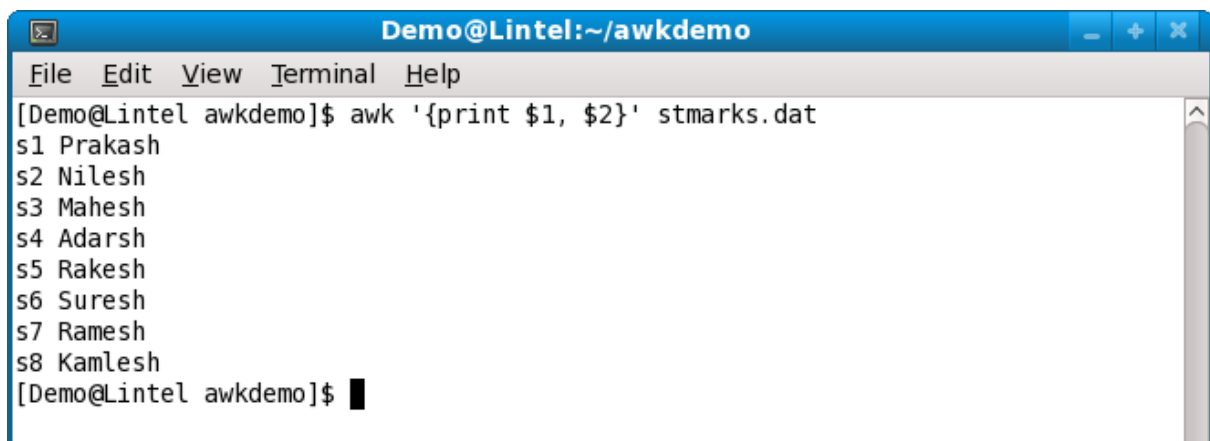
Let us see other few examples to understand the use of AWK. In the following example, we will consider a student record file. The content of file "stmarks.dat" are student no, name, marks in subject-1, marks in subject-2 and marks in subject-3. Each record is a line in the file, a space separates each field. The content of file as shown below:



```
Demo@Lintel:~/awkdemo
File Edit View Terminal Help
[Demo@Lintel awkdemo]$ cat stmarks.dat
s1 Prakash 52 81 62
s2 Nilesh 72 74 38
s3 Mahesh 72 51 81
s4 Adarsh 46 72 83
s5 Rakesh 72 51 42
s6 Suresh 81 78 57
s7 Ramesh 52 69 30
s8 Kamlesh 32 62 80
[Demo@Lintel awkdemo]$
```

Figure 3: Content of stmarks.dat file

To display the initial two fields of file "stmarks.dat", we use the print command as shown earlier. However, here the field separator is space so there is no need to explicitly specify the field separator using the "F" option. The following figure demonstrates use of print command with awk:



```
Demo@Lintel:~/awkdemo
File Edit View Terminal Help
[Demo@Lintel awkdemo]$ awk '{print $1, $2}' stmarks.dat
s1 Prakash
s2 Nilesh
s3 Mahesh
s4 Adarsh
s5 Rakesh
s6 Suresh
s7 Ramesh
s8 Kamlesh
[Demo@Lintel awkdemo]$
```

Figure 4: Using print command with awk

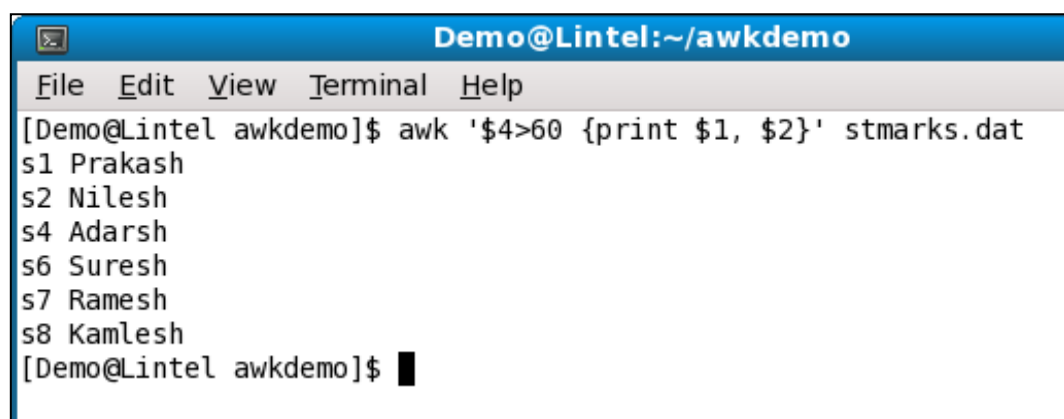
Using operators with AWK

AWK supports a wide range of operators. The following table lists various comparison operators supported by awk:

Expression	Result
------------	--------

<code>a < b</code>	True if a is less than b
<code>a <= b</code>	True if a is less than or equal to b
<code>a > b</code>	True if a is greater than b
<code>a >= b</code>	True if a is greater than or equal to b
<code>a == b</code>	True if a is equal to b
<code>a != b</code>	True if a is not equal to b
<code>a ~ b</code>	True if the string a matches the regexp denoted by b
<code>a !~ b</code>	True if the string a does not match the regexp denoted by b

The following screen-shot demonstrates an example where comparison operator is applied. In the following screen, the command lists the student number and student name who have scored marks above 60 in subject-1:



```

Demo@Lintel:~/awkdemo
File Edit View Terminal Help
[Demo@Lintel awkdemo]$ awk '$4>60 {print $1, $2}' stmarks.dat
s1 Prakash
s2 Niles
s4 Adarsh
s6 Suresh
s7 Ramesh
s8 Kamlesh
[Demo@Lintel awkdemo]$

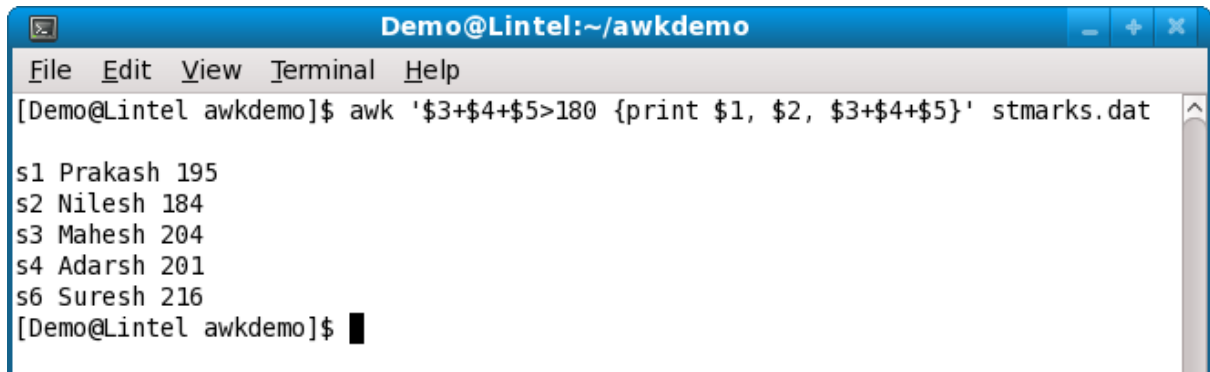
```

Figure 5: Using comparison operator in awk

Using Arithmetic Operators with AWK

AWK supports arithmetic operators like addition (+), subtraction (-), multiplication (*), division (/), remainder (%), exponentiation (^), increment (++) and decrement (--) with pattern matching.

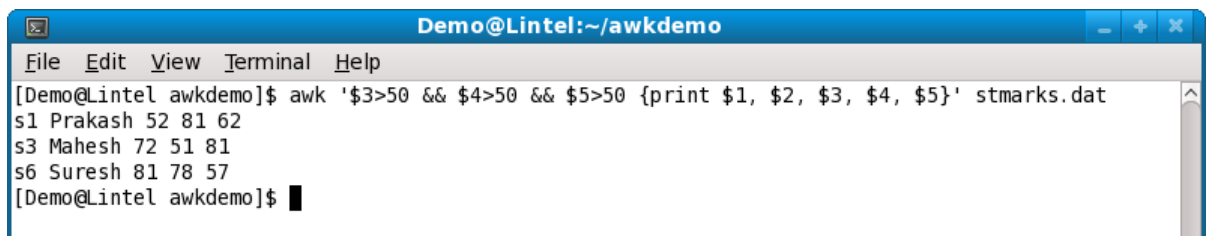
The following illustration demonstrates the use of addition operator to display those records where the total of marks of three subjects is above 180:



```
Demo@Lintel:~/awkdemo
File Edit View Terminal Help
[Demo@Lintel awkdemo]$ awk '$3+$4+$5>180 {print $1, $2, $3+$4+$5}' stmarks.dat
s1 Prakash 195
s2 Nilesh 184
s3 Mahesh 204
s4 Adarsh 201
s6 Suresh 216
[Demo@Lintel awkdemo]$
```

Figure 6: Using arithmetic operators with AWK

In the following figure, we have applied multiple conditions to selectively filter records. The following example demonstrates the display of only those records where the marks in 3 different subjects are above 50:

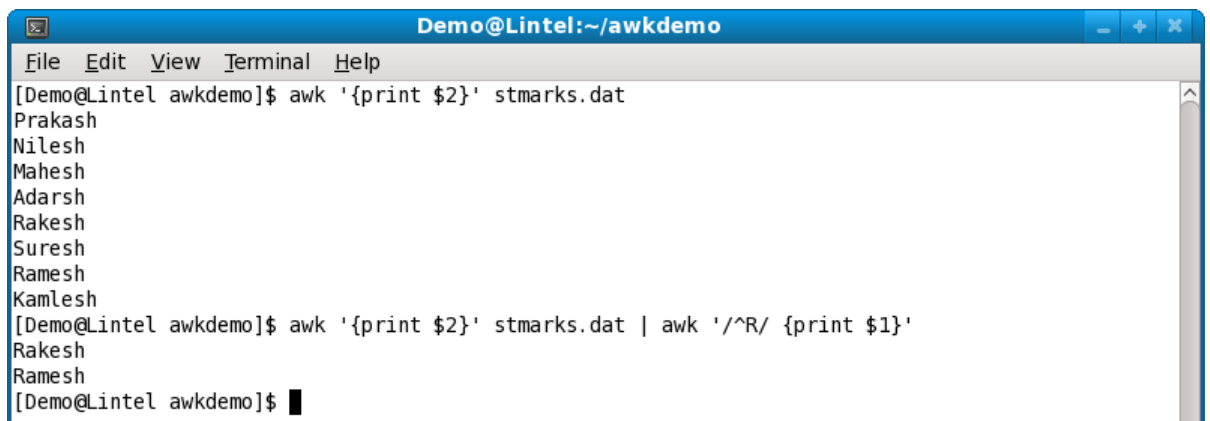


```
Demo@Lintel:~/awkdemo
File Edit View Terminal Help
[Demo@Lintel awkdemo]$ awk '$3>50 && $4>50 && $5>50 {print $1, $2, $3, $4, $5}' stmarks.dat
s1 Prakash 52 81 62
s3 Mahesh 72 51 81
s6 Suresh 81 78 57
[Demo@Lintel awkdemo]$
```

Figure 7: Comparison and && operator with AWK

Using regular expression with AWK

Let us see an example how regular expression can be used with AWK. In the following illustration, we are listing those records where the student name begin with „R“:



```
Demo@Lintel:~/awkdemo
File Edit View Terminal Help
[Demo@Lintel awkdemo]$ awk '{print $2}' stmarks.dat
Prakash
Nilesh
Mahesh
Adarsh
Rakesh
Suresh
Ramesh
Kamlesh
[Demo@Lintel awkdemo]$ awk '{print $2}' stmarks.dat | awk '/^R/ {print $1}'
Rakesh
Ramesh
[Demo@Lintel awkdemo]$
```

Figure 8: Using regular expression with AWK

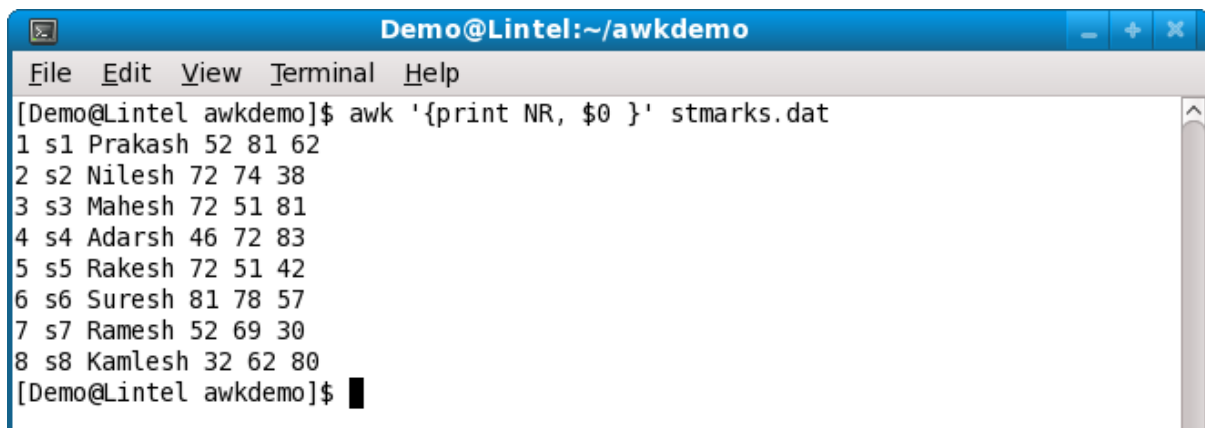
Using Built-in variables with AWK

AWK supports the use of both user-defined variables and built-in variables. In this section, let us explore the built-in variables supported by AWK. The built-in variables are used in AWK script

The following variables are supported by AWK:

- **NR:** Keeps count of the number of input records, indicates current record.
- **NF:** Keeps a count of the number of fields
- **FILENAME:** The name of the current input-file
- **FNR:** No of records in current filename
- **FS:** Contains the "field separator" character
- **RS:** Stores the current "record separator" or row Separator
- **OFS:** Stores the "output field separator"
- **ORS:** Stores the "output record separator"
- Built-in variables also include the field variables like \$1, \$2, \$3, and so on (where \$0 represents the entire record). These variables contain the values/text stored in the individual text-fields in a record.

Let us study how built-in variables can be used while processing records. In the following example, we are displaying the record count which is indicated as line number before the record:



```
Demo@Lintel:~/awkdemo
File Edit View Terminal Help
[Demo@Lintel awkdemo]$ awk '{print NR, $0 }' stmarks.dat
1 s1 Prakash 52 81 62
2 s2 Niles 72 74 38
3 s3 Mahesh 72 51 81
4 s4 Adarsh 46 72 83
5 s5 Rakesh 72 51 42
6 s6 Suresh 81 78 57
7 s7 Ramesh 52 69 30
8 s8 Kamlesh 32 62 80
[Demo@Lintel awkdemo]$
```

Figure 8: Using built-in variables with AWK

Creating AWK Scripts

AWK scripts are small programs that can perform multiple actions combined. Usually, the awk scripts are saved with .awk file extension. AWK script provides the facility of pre-processing and post-processing using the BEGIN and END keywords. For instance, if we want to print something before the input process starts, the BEGIN section can be used. Similarly, the END section is useful to

print something after the processing gets over. BEGIN and END sections are optional. The structure of AWK script is as follows:

- An optional **BEGIN** Statement
(used for processing that executes prior to reading input)
- Pattern – action pairs
(used for processing input data & matching patterns if any)
- An optional **END** segment
(used for processing after end of input data)

Let us see an example of AWK script to calculate the total marks of subject-3 for all students using the stmarks.dat file.

```
#!/usr/bin/awk -f
BEGIN {sum=0}
{sum=sum+$5}
END {print sum}
Usage:
$awk -f sample.awk stmarks.dat
```

In the given script, we process stmarks.dat file that was discussed earlier. The script is stored in sample.awk file. BEGIN section is used to initialize a user-defined „sum“ variable while the END section is used to display the content of sum variable.

BEGIN can be used to print headers as follows:

```
$awk „BEGIN{printf \"No\tName\tSub \tMarks\n\"} {print}\" stmarks.dat
```

Few other features of AWK

AWK supports usage of functions like length(), substr(), index(), split() etc to perform operations on strings. These functions can be used to perform string operations. Mathematical functions like sqrt(), etc. are also supported. AWK also supports if statement that is used for control flow. Looping structures like for and while are also supported by AWK. Looping can be used frequently when we want to process an array or string.

Summary

Let us summarize what we have studied in this module:

AWK is a programming language that has BEGIN-END format while scripting

AWK can also be used at command-prompt

AWK can process large volume of text-data and can be used for summarizing or reporting purposes
